# Tortoise Occupancy

Ellen Bledsoe

2026-04-06

## Desert Tortoise Occupancy

Analyze survey data for desert tortoises from Saguaro National Park

### Set-Up

I have installed the `unmarked` package for you already in Posit Cloud. This is a new package we will be using to do our occupancy calculations.

The code for using `unmarked` is not intuitive AT ALL. We will walk through an example together first, and then you can apply the code we write today to your assignment.

We will need to first load `unmarked` packages.

```
library(unmarked)
```

Now we need to read in our data set, called `tortoise_occupancy.csv`, using the `read.csv()` function.

```
# your filepath will not have "../data_raw/" in it
tort_occ <- read.csv("../data_raw/tortoise_occupancy.csv")
```

#### Explore the data

First, let's take a look at `tort_occ`, our tortoise occupancy data. What does each column represent? How about each row?

```
head(tort_occ)
```

```
##   j_1 j_2 j_3 j_4 j_5
## 1   1   1   1   1   1
## 2   0   0   1   0   0
## 3   0   0   0   0   0
## 4   1   0   1   0   0
## 5   0   0   0   0   0
## 6   1   1   0   0   0
```

### Naive Occupancy

Before we run any occupancy models that take detection probability into account, let's calculate the naive occupancy.

Naive occupancy is calculated by the number of sites where a species was detected (at least once) divided by the total number of sites surveyed.

```
# count up the number of times each site was marked as occupied
# returns a vector that we can now use
```

```r
num_detections <- rowSums(tort_occ)
num_detections
```

```
##  [1] 5 1 0 2 0 2 4 0 0 3 1 4 0 2 0 0 0 3 2 3
```

```r
# total number of sites that were surveyed
sites_surveyed <- nrow(tort_occ)
sites_surveyed
```

```
## [1] 20
```

```r
# number of sites with detection of greater than zero
sites_detection <- length(num_detections[num_detections > 0])
sites_detection
```

```
## [1] 12
```

```r
# divide the number of sites with detection
naive_occupancy <- sites_detection / sites_surveyed
naive_occupancy
```

```
## [1] 0.6
```

Is this estimate of occupancy going to be higher or lower than when we take detection probability into account?

## Occupancy Models

Now we are going to run a few models that take detection probability into account in our estimate. To do this, we will use the `unmarked` package, which has functions specifically for running these types of models.

To run correctly, the `unmarked` package requires we put our data in a very specific format.

### Prepare Our Data

**Create a time data frame (Observation co-variates)** First, we need to create a list of data frames for our observation co-variates (meaning, variables that might change with each observation).

This list of data frames (in our case, just 1 data frame) is what will allow us to create a model where detection probability varies with time (each survey occasion).

```r
# create the data frame with each column with the survey number for 20 rows
surveys_df <- data.frame(time_1 = rep("1", 20),
                         time_2 = rep("2", 20),
                         time_3 = rep("3", 20),
                         time_4 = rep("4", 20),
                         time_5 = rep("5", 20))

# make this into a list of one data frame
surveys_list <- list(time = surveys_df)
```

**Create `unmarked` data frame** Now that we have both our occupancy data frame (the encounter histories) and a list that contains our survey data frame, we can create the specific type of data frame that the `unmarked` package requires.

To do this, we use the `unmarkedFrameOccu()` function.

Running this line of code will produce a warning, but you can ignore it.

```r
# set the obsCovs argument equal to the LIST made above
# ERROR means something went wrong; WARNING is fine
tortUMF <- unmarkedFrameOccu(tort_occ, obsCovs = surveys_list)
```

Let's take a look at this new data frame that we just created.

```r
head(tortUMF)
```

```
## Data frame representation of unmarkedFrame object.
##   y.1 y.2 y.3 y.4 y.5 time.1 time.2 time.3 time.4 time.5
## 1   1   1   1   1   1      1      2      3      4      5
## 2   0   0   1   0   0      1      2      3      4      5
## 3   0   0   0   0   0      1      2      3      4      5
## 4   1   0   1   0   0      1      2      3      4      5
## 5   0   0   0   0   0      1      2      3      4      5
## 6   1   1   0   0   0      1      2      3      4      5
```

```r
summary(tortUMF)
```

```
## unmarkedFrame Object
##
## 20 sites
## Maximum number of observations per site: 5
## Mean number of observations per site: 5
## Sites with at least one detection: 12
##
## Tabulation of y observations:
##  0  1
## 68 32
##
## Observation-level covariates:
##  time
##  1:20
##  2:20
##  3:20
##  4:20
##  5:20
```

### Fit Models for Occupancy

Occupancy models take the data from our encounter histories and first estimates detection probability. It then uses that detection probability estimate to produce an estimate of occupancy (our parameter of interest).

The function to run the occupancy model in `unmarked` is `occu()`. The function always take the argument for detection probability first and then the argument for occupancy.

Let's demonstrate. Our first model will have detection probability consistent across time (surveys) and occupancy consistent across sites. Remember, an assumption we have about occupancy is that it stays consistent through our surveys.

When we are keeping an estimate constant, we use `~1`.

```r
# Model p(.)psi(.) #
# detection prob (p) constant across survey occasion,
# occupancy (psi) constant across sites,
pdot_psidot  <- occu(~1 ~1, data = tortUMF)
```

Our second model will have occupancy consistent across sites but detection variability can vary with time (survey occasion). To indicate this, we put `~time-1` in the first spot for detection probability.

```
# Model p(t)psi(.) #
# detection prob (p) varies with time (survey occasion)
# occupancy (psi) constant across sites
ptime_psidot <- occu(~time-1 ~1, data = tortUMF)
```

### Model Selection

We can run both models and then use AIC (an index of model quality) to determine which model fits our data the best. We want to choose the model with the lowest AIC because it represents the most parsimonious model.

- We first make an object (`models`) with a list of our models using a function called `fitList()`.
- Then we have the function `modSel()` run the models and present us with the AIC values.
- We then choose the model with the *lowest* AIC value (looking at the AIC column).

```
# Assemble models into a list
models <- fitList(pdot_psidot  = pdot_psidot,  # name in output = object
                  ptime_psidot = ptime_psidot) # name in output = object

# Use modSel() to calculate and compare their AIC values
modSel(models)
```

```
##              nPars    AIC delta AICwt cumltvWt
## pdot_psidot      2 113.25  0.00  0.75     0.75
## ptime_psidot     6 115.49  2.23  0.25     1.00
```

Based on the AIC values, which is our top model?

### Estimates & 95% CIs

Now that we have chosen our top model, we want to discover what the actual values detection probability (p) and occupancy (psi) are and our uncertainty around them (95% confidence intervals).

Thankfully, there are functions that produce these values for us.

First, let's look at the parameter estimates (p and psi).

```
# This returns the coefficients (estimates) for the model
# coef() function returns model coefficients (estimates)
# plogis() coverts the values from a logistic scale to probabilities

# For model: p(.)psi(.)
plogis(coef(pdot_psidot))
```

```
##  psi(Int)    p(Int)
## 0.6157371 0.5197018
```

```
# For model: p(t)psi(.)
plogis(coef(ptime_psidot))
```

```
##  psi(Int)  p(time1)  p(time2)  p(time3)  p(time4)  p(time5)
## 0.6112910 0.7361347 0.3271639 0.6543438 0.4089494 0.4907800
```

Note that there are 5 estimates for detection probability in the model that we allowed to vary through time. This is because the model produced a detection probability estimate for each survey occasion rather than just one overall detection probability estimate.

We can also calculate the 95% confidence intervals for each estimate (`level = 0.95`). This time we use the `confint()` function. We need to indicate the estimate type:

- detection probability: `type = "det"`
- occupancy: `type = "state"`, as in the "true state" of occupancy

```r
# Interpreting the Output
# Column 0.025 -> Lower Bound for 95% CI
# Column 0.975 -> Upper Bound for 95% CI

# model p(.)psi(.)
plogis(confint(pdot_psidot, type = "det",   level = 0.95)) # 95% CI for p
```

```
##            0.025     0.975
## p(Int) 0.3867842 0.6498879
```

```r
plogis(confint(pdot_psidot, type = "state", level = 0.95)) # 95% CI for psi
```

```
##              0.025     0.975
## psi(Int) 0.3858832 0.8033929
```

```r
# model p(t)psi(.)
plogis(confint(ptime_psidot, type = "det",   level = 0.95))
```

```
##              0.025     0.975
## p(time1) 0.4295640 0.9117811
## p(time2) 0.1276352 0.6177361
## p(time3) 0.3625520 0.8630293
## p(time4) 0.1799118 0.6857501
## p(time5) 0.2368970 0.7495098
```

```r
plogis(confint(ptime_psidot, type = "state", level = 0.95))
```

```
##              0.025     0.975
## psi(Int) 0.3843875 0.7984203
```

Stop for a moment and think about what the `0.025` and `0.975` values represent. These are *not* estimates. Instead, they are signalling the *lower* and *upper* bounds of the confidence intervals, respectively.

Remember how confidence intervals are calculated in part based on a t-value from the Student's t-distribution? Each tail of the distribution represents 2.5% of the remaining probability, meaning the 95% confidence intervals include the values between 2.5% and 97.5% of the distribution.

Similar to the parameter estimates, we have 5 sets of confidence intervals for the model that allows the estimates of detection probability to vary with time; these are the confidence intervals for each estimate.